

ARK-460 — Preregistration

Cloud IAM Role Grant · Dependency Loss

Series: ExecutionProof authorization-boundary corpus — **Production-Boundary phase (P01)**

Experiment ID: ARK-460

Institution: Remnant Fieldworks Inc.

PI: Derek Adam Hone

Substrate: Classical software (no quantum hardware, no cryptographic security claims)

Guards: Dual independent — V1 (JavaScript), V2 (Python)

Preregistration status: LOCKED before official execution (see `MANIFEST.txt`)

Commercial / standards / IP justification

- **Commercial purpose:** Answers **Prospect Question #5** — “What happens when a critical dependency is down?” — in the AWS-style IAM context. Production systems depend on the Policy Registry, Identity Provider, Signature Verifier, and Audit Logger. When any is **UNAVAILABLE** at execution time, a correct gate must **fail safe** (HOLD — cannot confirm validity) rather than wrongly ALLOW (fail-open, a catastrophic breach).
 - **RF-100 requirement:** Exercises the **fail-safe-on-dependency-loss** conformance requirement — that an approval does not grant perpetual authority even when dependencies are down; the gate must HOLD if it cannot confirm validity.
 - **Patent-family connection:** Produces dated **working-example** evidence for the dependency-loss / fail-safe mechanisms. Working-example evidence only — it does not legally validate any claim, and adds no new matter to any filed application.
-

Question

When an IAM role-grant authorization is **APPROVED**, but a critical dependency (Policy Registry, Identity Provider, Signature Verifier, Audit Logger) is **UNAVAILABLE** at execution time, does the gate:

- **HOLD** when any dependency is unavailable (fail-safe — cannot confirm validity)?
- **ALLOW** only when **all** dependencies are available (can confirm validity)?

The IAM action itself is exact-match (from ARK-458); only **dependency availability** varies.

Model

Each scenario has:

- **IAM action tuple** (from ARK-458, exact-match on all 5 dimensions)
- **dependencies** dict: `{"PolicyRegistry": "AVAILABLE"|"UNAVAILABLE", "IdentityProvider": ..., "SignatureVerifier": ..., "AuditLogger": ...}`

Decision logic:

- If **ALL** critical dependencies are AVAILABLE → **ALLOW** (can confirm validity)
- If **ANY** critical dependency is UNAVAILABLE → **HOLD** (cannot confirm validity, fail-safe)

Arms

8 arms × 100 scenarios per arm = **800 evaluation decisions**.

Arm	Condition	Expected
1	ALL-AVAILABLE	ALLOW
2	PolicyRegistry UNAVAILABLE	HOLD
3	IdentityProvider UNAVAILABLE	HOLD
4	SignatureVerifier UNAVAILABLE	HOLD
5	AuditLogger UNAVAILABLE	HOLD
6	MULTIPLE UNAVAILABLE (≥ 2)	HOLD
7	ALL-UNAVAILABLE (total outage)	HOLD
8	ALL-AVAILABLE (baseline recheck)	ALLOW

Dependency-gate oracle

An **independent structural oracle** (`_dependency_gate_oracle`) verifies, for every generated scenario, that it genuinely encodes its arm's dependency state before any guard sees it. Any violation ⇒ abort.

Metrics

- **g_allow_available** = ALLOW rate on arms {1, 8} (all dependencies available)
- **g_hold_unavailable_min** = min HOLD rate across arms {2, 3, 4, 5, 6, 7} (at least one dependency unavailable)
- **g_margin** = $\min(\text{g_allow_available}, \text{g_hold_unavailable_min}) - 0.05$
- Secondary: V1↔V2 decision concordance (target 100%).

Pass criteria (fixed before execution)

- **C1:** `g_allow_available` ≥ 0.95
- **C2:** `g_hold_unavailable_min` ≥ 0.95

- **C3:** `g_margin` ≥ 0.90
- **Verdict** = **PASS** iff $C1 \wedge C2 \wedge C3$, else **FAIL**.

Kill conditions / effectiveness gate

1. **Effectiveness oracle:** every scenario must genuinely encode its arm's dependency state. Any violation $\Rightarrow$ abort.
2. **Kill-gate (negative control):** a deliberately broken guard that **ignores dependency availability** and always **ALLOWS** must wrongly **ALLOW** $> 50\%$ of the **HOLD** arms $\{2,3,4,5,6,7\}$ — proving the testbed is falsifiable.

Publication rule

The outcome (PASS, FAIL, or ABORT) will be published to GitHub with raw per-arm results, and defensively published to Zenodo (CC BY 4.0), regardless of verdict.

Honest bounds

- Classical software test of **authorization control logic**, not a test of AWS or any real dependency infrastructure, not a cryptographic security proof, not a production-readiness certification.
- This tests the **dependency-loss logic**; ARK-458 tested exact-action-binding, ARK-459 tested revocation.
- Results are bounded to this scenario model, these arms, and this seed family.
- No claim that this experiment legally validates any patent claim or certifies RF-100 conformance.

Preregistered under the Remnant Fieldworks Standing Covenant (preregister $\rightarrow$ lock $\rightarrow$ execute $\rightarrow$ publish all outcomes). To God be the glory. Proof Before Power. Verification Before Execution.